Technical Approach Report: AI PR Review Assistant

Prepared for: AI/ML Engineer Intern assessment

Author: Yuvraj Rathore

Repository: https://github.com/yuvrajrathore672/Cynbit_PR_review_assistant

Live Demo: https://pr-assistant.streamlit.app/

---

1. Problem and objective

BitElevate, Cynbit's 60-day remote internship program, asks every student to submit a pull request (PR) each day, and senior mentors review each PR line by line. This is a core part of the program's value, but it is also its scaling bottleneck: mentor time grows linearly with the number of students.

Objective: build a first-pass AI reviewer that reads a PR diff and drafts structured feedback, so a mentor starts from a reviewed draft instead of a blank page. The AI does not replace the mentor. The mentor approves or edits every comment before the student sees it (human-in-the-loop).

2. System architecture

```
PR diff (pasted text or file)
        |
        v
  Preprocess: truncate very long diffs (12,000 characters)
        |
        v
  Prompt: mentor-style system prompt + diff
        |
        v
  LLM (openai/gpt-oss-120b on Groq, JSON mode, temperature 0.1)
        |
        v
  Validate with Pydantic schema --(invalid)--> retry with the error message (max
2 attempts)
        |
        v
  ReviewResult: summary, issues[], rubric scores, mentor comment
        |
        v
  Streamlit UI: scores, issue cards, editable mentor comment
```

Components

File	Role
`reviewer/core.py`	`review_diff()`: builds messages, calls the model, validates, retries
`reviewer/prompt.py`	System prompt that defines the reviewer role, rules and JSON shape

File	Role
`reviewer/schemas.py`	Pydantic models (`Issue`, `RubricScores`, `ReviewResult`)
`app.py`	Streamlit demo UI
`scripts/evaluate.py`	Evaluation against planted flaws
`data/`	Sample diffs and the expected-issues answer key
`tests/`	pytest checks for data integrity and schema validation

3. Model choice

- **Model:** `openai/gpt-oss-120b`, an open-weight 120B-parameter model, served through the **Groq API**.
- **Why:** Groq gives fast inference and a free tier, which suits rapid prototyping. The model is large enough to reason about code, and Groq's JSON mode supports structured output.
- **Framework:** LangChain's `ChatGroq` wrapper. LangChain is part of Cynbit's stated stack, and it makes it easy to swap providers later.
- **Swappable by design:** the model name is read from the `GROQ_MODEL` environment variable, so changing models requires no code change.
- **Why not fine-tune?** There is no labeled PR-review dataset yet, and a well-prompted general model already works for a first version. Fine-tuning is a later option once real mentor feedback has been collected (see section 9).
- **Trade-offs:** an external API means code leaves the company's environment, and there is rate-limit and availability risk. For production, a self-hosted open model or an enterprise API agreement would address privacy.

4. Prompt engineering

The system prompt does the following:

1. **Defines a role:** a senior mentor reviewing a student's PR in an internship.
2. **Limits scope:** review only the changed code, and check five categories: security, performance, testing, correctness and style.
3. **Reduces hallucination:** "report only real problems visible in the diff", and return an empty issues list for a good PR.
4. **Forces specificity:** every issue needs a file, a line if known, a severity, an explanation and a concrete fix.
5. **Controls tone:** the mentor comment must mention one good thing first, then the most important fixes, which suits students.
6. **Fixes the output shape:** the exact JSON structure and the allowed values for `severity` and `category` are given in the prompt.

Temperature is set to 0.1 to keep outputs consistent and low in randomness.

5. Structured output and validation

Free-text answers are hard for software to use. The pipeline therefore uses three layers:

1. **Prompt:** describes the exact JSON shape.
2. **JSON mode:** the provider is asked to return a valid JSON object.

3. **Pydantic validation:** every field is type-checked. `severity` and `category` use `Literal` types, and each rubric score must be an integer from 1 to 5. If validation fails, the error message is sent back to the model and it is asked to correct itself (up to 2 attempts).

This keeps malformed model output from reaching the UI or the evaluation. A pytest case confirms that an invalid severity value (for example `"scary"`) is rejected.

6. Data

No model training is involved, so the data requirement is small:

- **5 synthetic PR diffs, each with one planted flaw:** a hardcoded API key, a missing database index on a foreign key, JWT signature verification disabled, new functions without tests, and SQL built with string formatting (injection).
- **1 clean PR** (code plus tests), to check for false alarms.
- **`expected_issues.json`:** the answer key, with a category and keywords for each planted flaw.

The diffs are modelled on the kinds of mistakes BitElevate students describe in their own testimonials (for example, missing foreign-key indexes and insecure JWT handling). All data is synthetic, so no student code or personal data is used.

7. Evaluation

Method: run the reviewer on each diff. A planted flaw counts as **caught** if the model reports an issue with the matching category and at least one expected keyword in its title, explanation or fix. On the clean PR, any issue the model reports as `critical` or `major` counts as a **false alarm**. The metric is recall (caught divided by planted).

Results (two full evaluation runs on the 6 diffs):

	Recall	False alarms on clean PR	Missed flaw
Run 1	4/5 (0.80)	0	missing tests (`04_no_tests`)
Run 2	4/5 (0.80)	0	missing index (`02_missing_index`)

Findings

- Every planted flaw was caught in at least one run, and the clean PR produced no critical or major false alarms.
- **Results vary between runs.** The two misses were different flaws, and a separate manual run on `04_no_tests` did find the missing tests. Single-run scores are therefore noisy, even at low temperature.
- The model also reported extra issues beyond the planted ones (for example missing docstrings and input validation). Some are useful, and some are minor. The metric measures recall only, so these extras are not scored.
- During evaluation the provider returned a "JSON validation failed" error once, when the model wrapped part of its JSON in markdown bold characters. This confirms that retries and validation are needed in production, and the evaluation script retries on such failures.

8. Limitations

1. **Tiny dataset:** 6 diffs is a proof of concept, not a benchmark.
2. **Synthetic data:** real student PRs are longer, messier and involve more files.

3. **Keyword-based scoring:** category plus keyword matching is an approximation. An LLM judge or human labels would be more reliable.
4. **No precision metric:** extra issues are not checked for correctness.
5. **Run-to-run variance:** a single review can miss a real issue.
6. **Diff-only context:** the model sees only the diff and not the whole repository, so it cannot judge things like existing tests elsewhere.
7. **Truncation:** diffs over 12,000 characters are cut, so large PRs are partially reviewed.

9. Cost, privacy and risk

- **Cost:** the prototype runs on a free tier. At scale, cost is roughly (PRs per day) x (tokens per diff and review). Diff truncation and a smaller model for easy cases keep this predictable.
- **Privacy:** student code is sent to an external API. Students should be told about this, and a self-hosted model is the longer-term option.
- **Safety and trust:** every AI comment passes through a mentor before reaching a student. The AI is described as an assistant, not as the reviewer of record.
- **Failure handling:** invalid model output is retried, and the UI shows a clear error if a review fails.